Two Brains, One Network: Dual-Process Topology Management for Heterogeneous Physics-Informed Kolmogorov-Arnold Networks

Richard Liu Yufei Zou
School of Data Science
University of North Carolina at Charlotte
github.com/RicardoLaMo/OpKAN

March 2026

Abstract

Physics-informed Kolmogorov-Arnold Networks (KANs) introduce a structural management problem that gradient descent cannot solve: when should an edge remain a B-spline, be replaced by a symbolic function, or be pruned? We formalize this as the *topology surgery problem* and show that it creates five chronic failure modes in heterogeneous KANs: topology stagnation, parameterization lock-in, symbolic-numerical tension, regime blindness, and instability in the second-derivative smoothness required by the PDE. To address this, we propose a **dual-process topology agent**: a fast reflexive controller performs high-frequency local maintenance such as pruning and learning-rate stabilization, while a slow strategic controller performs low-frequency topology surgery guided by PDE geometry and market-regime signals. A typed DSL, static AST allowlist, and atomic rollback ensure that every mutation preserves twice-differentiable smoothness (C^2 continuity) and transactional integrity. We demonstrate the framework on a Heston stochastic-volatility PI-KAN, where stable second derivatives are a hard requirement because of the cross-derivative term. An ablation shows that System 2 is required for regime-ordered behavior, while System 1 restores full throughput; PDE residual minimization alone fails all five financial monotonicity checks under distributional shift, revealing a broader training-objective insufficiency in physics-constrained learning under regime change. The system sustains **26,300** smp/sec on H200 with zero GPU blocking, showing that online structural governance need not compromise training efficiency.

1 Introduction

1.1 Problem: The Topology Surgery Problem

A Kolmogorov-Arnold Network (KAN) places learnable activation functions on *edges* rather than nodes, which means the network is a mutable mathematical object: any edge can be a B-spline, replaced by a closed-form symbolic function, or pruned to zero during training. This edge-level replaceability is what makes KANs attractive for physics-informed settings—it allows domain knowledge to be injected directly into the network's mathematical structure.

However, it also creates a problem that gradient descent cannot solve. Gradient descent can optimize the coefficients of a B-spline edge, but it cannot decide that the edge should *switch class*: from spline to symbolic form, or from a weak edge to zero. A near-zero activation has a near-zero gradient, so the edge persists indefinitely consuming computation. A B-spline that happens to approximate $\sin(x)$ remains a B-spline indefinitely, losing interpretability and efficiency. We call this the **topology surgery problem**: a class of structural decisions that falls strictly outside

the scope of gradient-based optimization.

This problem manifests in five distinct failure modes, each arising from a specific gradient-signal gap—a setting where the training loss provides no corrective signal for the structural decision that must be made:

1. **Topology stagnation.** Near-zero B-spline edges persist through training because their loss gradient is also near-zero. The edge contributes nothing but continues consuming computation.
2. **Parameterization lock-in.** Gradient descent has no incentive to change an edge's parameterization class. A B-spline approximating $\sin(x)$ remains a B-spline indefinitely, losing interpretability and efficiency.
3. **Symbolic-numerical tension.** Symbolic edges have no trainable parameters. In a mixed topology, gradient cannot flow through symbolic edges to inform neighboring B-spline adaptations, creating dead zones in the optimization landscape.
4. **Regime blindness.** Under distributional shift (market regime change), a frozen topology cannot adapt. Weight updates shift the learned function but cannot restructure which edges carry which mathematical primitives.
5. **C^2 instability.** Aggressive learning-rate updates can make an edge insufficiently smooth for stable second-derivative computation. In the Heston setting, the PDE contains a cross-derivative term $\rho\sigma v S\, \partial^2 V/\partial S \partial v$ evaluated by autograd, so unstable edge curvature propagates as NaN.

1.2 Our Approach: Dual-Process Topology Agent

The five problems decompose naturally by frequency and cognitive type. Problems 1 and 5 (topology stagnation and C^2 instability) are high-frequency and low-complexity: they can be detected from local statistics and handled with simple actions such as pruning or learning-rate adjustment. These are reflexive maintenance tasks; a fast model (<50 ms, 7B parameters) is sufficient.

Problems 2, 3, and 4 (parameterization lock-in, symbolic-numerical tension, and regime blindness) are low-frequency and high-complexity. They require reasoning about global activation geometry, the mathematical validity of candidate symbolic forms, and the relationship between market regime and topology adaptation. A slower, larger model (200–500 ms, 32B parameters) is needed for this multi-step mathematical reasoning.

A single fast model makes mathematically incoherent symbolic replacements. A single slow model blocks the GPU. The **dual-process design** is not merely an efficiency tradeoff; it follows from the structure of the problem itself. Reflexive maintenance and strategic topology surgery are different kinds of decisions that should not be collapsed into a single controller.

We evaluate this framework on a Heston stochastic-volatility PI-KAN, where C^2 continuity, PDE geometry reasoning, and real-time regime adaptation impose hard correctness requirements that expose dual-process advantages invisible in unconstrained settings.

1.3 Contributions

1. A taxonomy of five chronic management problems in heterogeneous KAN topology, each mapped to a specific gradient-signal gap, and a demonstration that they decompose across two cognitive timescales.
2. A **dual-process topology agent** (System 1/System 2) that addresses all five failure modes, with the two paths operating concurrently without GPU blocking.
3. A **typed DSL** with AST allowlist and atomic rollback providing type safety, vocabulary restriction (C^2 continuity preserved by construction), and transactional integrity.
4. **Evaluation on Heston options pricing** with the key finding that PDE residual min-

imization alone fails all five financial monotonicity checks under distributional shift—a training-objective insufficiency result that may extend to other physics-constrained networks.

2 Related Work

KANs and symbolic regression. The KAN architecture [Liu et al., 2024] places learnable activations on edges, grounded in Kolmogorov's superposition theorem [Kolmogorov, 1957]. Symbolic edge replacement exists in the original library but is manual and offline; our contribution is making it online and agent-driven under formal invariants.

Physics-informed neural networks and finance. PINNs [Raissi et al., 2019] embed differential operator residuals in the training loss. Financial applications [Horvath et al., 2021] have used MLP backbones; the C^2 continuity required by the Heston cross-derivative term has received limited attention, motivating our B-spline edge design.

LLM agents and dual-process AI. Kahneman's fast/slow framework [Kahneman, 2011] motivates the two-speed design. Toolformer [Schick et al., 2023] and ReAct [Yao et al., 2023] use LLMs as tool orchestrators over stateless API calls; our agent manages the continuous evolution of a heterogeneous mathematical object under formal invariants, a fundamentally different task.

Regime detection. Hamilton's HMM [Hamilton, 1989] is our backbone for market regime identification. The regime-blindness failure mode (Problem 4) shows why a topology agent must be regime-aware; PDE loss alone does not impose behavioral ordering across Heston parameter regimes.

Neural architecture search. NAS [Elsken et al., 2019] automates topology discovery offline. Our agent operates online, on a running network, with natural language input and domain-specific mathematical constraints—complementary to NAS for initialization, but a distinct runtime concern.

3 Methods

3.1 Data

Options data. We use OPRA options data for SLV (iShares Silver Trust ETF). The live benchmark uses a single quote date for cross-sectional evaluation; the replay benchmark replays a historical sequence of silver/regime feature data with a replayed SLV surface template to evaluate performance across regime transitions over time.

HMM features. The regime detector consumes five time-series features computed from market microstructure: (1) log-return ℓ_t, (2) realized volatility $\hat{\sigma}_t$ (20-bar window, annualized), (3) implied volatility σ_t^{IV}, (4) IV–RV spread $\Delta_t = \sigma_t^{\mathrm{IV}} - \hat{\sigma}_t$, and (5) IV momentum $\mu_t^{\mathrm{IV}} = \sigma_t^{\mathrm{IV}} - \sigma_{t-1}^{\mathrm{IV}}$.

Collocation points for PDE training. The PI-KAN is trained on randomly sampled collocation points (S, v, t) drawn uniformly from $[0, 200] \times [0, 0.5] \times [0, 1]$ and normalized to $[0, 1]^3$ before being passed to the network. Interior points are used to evaluate the Heston PDE residual; boundary points are used to enforce the terminal payoff and boundary conditions.

Model families evaluated. Six model configurations are compared: `bs_plain` (Black-Scholes with baseline ATM volatility), `bs_regime` (Black-Scholes with HMM regime-predicted volatility), `rf_direct` (random forest direct price prediction), `spikan_hybrid` (symbolic PI-KAN residual + regime-vol BS anchor), `spikan_plain_hybrid` (symbolic PI-KAN residual + plain-formula BS anchor), and `spikan_plain_iv_hybrid` (plain-formula anchor + learned IV-space residual, our headline model).

3.2 Physics-Informed KAN (PI-KAN)

Why KAN over MLP. Standard MLP activations (ReLU) produce zero second derivatives, making the Heston cross-derivative term $\rho\sigma v S\, \partial^2 V/\partial S \partial v$ numerically degenerate when evaluated via autograd. KAN's B-spline edges—cubic splines with `grid_size=5` and `spline_order=3`—are C^2 continuous by construction, providing well-defined second derivatives throughout training.

The Heston PDE. Option fair value $V(S, v, t)$ satisfies:

$$\frac{\partial V}{\partial t} + \frac{1}{2} v S^2 \frac{\partial^2 V}{\partial S^2} + \rho\sigma v S \frac{\partial^2 V}{\partial S \partial v} + \frac{1}{2}\sigma^2 v \frac{\partial^2 V}{\partial v^2} + rS\frac{\partial V}{\partial S} + \kappa(\theta - v)\frac{\partial V}{\partial v} - rV = 0 \tag{1}$$

with boundary conditions: $V(S, v, T) = \max(S - K, 0)$ (terminal payoff), $V(0, v, t) = 0$ (zero spot), and $V(S, v, t) \approx S - Ke^{-r(T-t)}$ as $S \to \infty$.

Training loss. The composite loss minimizes PDE residual plus boundary error:

$$\mathcal{L} = \underbrace{\mathbb{E}_{\boldsymbol{x}\sim\Omega}\Big[|\mathcal{N}[V](\boldsymbol{x})|^2\Big]}_{\mathcal{L}_{\text{PDE}}} + \lambda_{\text{BC}} \underbrace{\mathbb{E}_{\boldsymbol{x}\sim\partial\Omega}\Big[|V(\boldsymbol{x}) - V_{\text{target}}(\boldsymbol{x})|^2\Big]}_{\mathcal{L}_{\text{BC}}} \tag{2}$$

where $\mathcal{N}[V]$ is the Heston differential operator evaluated via `torch.autograd` with `create_graph=True`.

Architecture. OpKAN uses a $[3, 16, 1]$ configuration (inputs (S, v, t), 16-node hidden layer, scalar output V). Three edge types coexist at runtime: `BSplineEdge` (8 basis functions, trainable coefficients + SiLU base), `C2SymbolicEdge` (closed-form expression from the 20-function AST allowlist, no trainable parameters), and `ZeroEdge` (returns `torch.zeros_like(x)`, implementing correct prune semantics).

3.3 Hidden Markov Regime Detector

A Gaussian HMM with $K = 3$ states and diagonal covariance is fit on the five market microstructure features above. Walk-forward inference uses non-overlapping 20-day windows, refitting from scratch at each boundary. The HMM produces regime label $r_t \in \{0, 1, 2\}$ at each step, where states are sorted by mean realized volatility to ensure label 0 always maps to the lowest-volatility regime across rolling windows (label stability fix). This label feeds System 2's context for strategic topology decisions. A detected regime transition triggers a strategic query even if fewer than 100 training steps have elapsed.

3.4 Dual-Process Topology Agent

The five failure modes decompose by cognitive timescale. Problems 1 and 5 (stagnation, C^2 instability) are high-frequency: detectable from local statistics, resolvable with threshold rules. Problems 2, 3, and 4 (parameterization lock-in, symbolic-numerical tension, regime blindness) are low-frequency but require multi-step mathematical reasoning over global network structure and PDE geometry.

System 1 (Reflexive, <50 ms, Qwen 2.5-7B). Every 10 training steps, the training loop places (step index, per-edge L1 norms, recent loss delta) into a bounded queue (`maxsize=1`). A background thread invokes the 7B model, which produces a `ReflexDecision`:

```
class ReflexDecision(BaseModel):
    reasoning: str        # brief justification
    prunes: List[str]     # edge IDs with L1 norm below threshold
    lr_adjustment: float  # multiplier in [0.5, 2.0]
```

The 7B model handles pattern recognition (is this norm below 0.005?), not deep reasoning. This path addresses Problems 1 (stagnation) and 5 (C^2 instability).

System 2 (Strategic, 200–500 ms, Qwen 2.5-32B). Every 100 training steps, a longer-horizon context (loss trajectory, HMM transition probabilities, full KAN topology state, regime signals) is placed in a separate bounded queue. A second background thread invokes the 32B model, which produces a `StrategicDecision`:

```
class StrategicDecision(BaseModel):
    reasoning: str                      # structural reasoning over PDE
        geometry
    mutations: List[EdgeMutation]       # REPLACE, PRUNE, or KEEP per
        edge
    regime_analysis: RegimeThesis       # HMM transition diagnosis
    training_command: Literal["CONTINUE", "HALT"]
```

The 32B model is necessary because edge replacement decisions require: (a) identifying that an activation profile approximates a specific smooth function, (b) verifying the candidate formula is C^2 on its natural domain, and (c) reasoning about how the replacement interacts with the Heston cross-derivative term. These are multi-step mathematical tasks that 7B models perform unreliably. This path addresses Problems 2, 3, and 4.

Non-blocking coordination. The `EngineCoordinator` manages four bounded queues (two input queues, `maxsize=1`; two output queues, `maxsize=32`). At the start of each training batch, `apply_pending_mutations()` non-blockingly drains both output queues. If no decisions are pending, training continues at full GPU throughput. The GPU never waits on the LLM.

3.5 Safety: Typed DSL and Atomic Rollback

The code injection problem. A naive design would pass LLM-generated formula strings directly to `eval()`, allowing arbitrary code execution. This is unacceptable for a system modifying live neural network weights.

Typed DSL with Pydantic validation. Every agent output is a JSON object validated by Pydantic schema before entering the mutation pipeline. This enforces structural well-formedness: required fields, type correctness, and value ranges.

AST allowlist. Formula strings within `EdgeMutation.formula` undergo static AST analysis. Only 20 `torch.*` functions are permitted, all of which are twice-differentiable by construction:

```
_ALLOWED_TORCH_FUNCS = frozenset({
    'sin', 'cos', 'tan', 'exp', 'log', 'log1p', 'sqrt', 'abs',
    'pow', 'tanh', 'sigmoid', 'softplus', 'sign', 'clamp',
    'relu', 'floor', 'ceil', 'round', 'reciprocal', 'neg'
})
```

The AST validator walks the full parse tree and raises `ValueError` for any node type not in the explicit allowlist, rejecting imports, file I/O, subscript access, and all other injection paths. The allowlist simultaneously enforces security and C^2 correctness.

Atomic rollback. Before applying any mutation batch, all affected edge modules are snapshotted via `copy.deepcopy`. If any exception occurs, all objects are restored from the snapshot. This uses module-level snapshots rather than `state_dict` because `swap_edge` replaces entire `nn.Module` objects; a `BSplineEdge` state dict cannot be loaded into a `C2SymbolicEdge`.

4 Results

4.1 Agent Ablation: Division of Labor

Table 1 confirms the division of labor. Monotonicity is entirely driven by System 2's strategic reasoning over PDE geometry and regime signals: neither gradient-only training nor System 1

Table 1: Monotonicity check pass rate and training throughput by agent configuration. No-agent baseline = pure gradient descent. The ablation reveals a clean division of labor: System 2 is required for regime-ordered behavior; System 1 recovers the throughput cost.

Agent Configuration	Monotonicity Checks Passed	Throughput
No agent (gradient only)	0 / 5	25,170 smp/sec
System 1 only (reflexive)	0 / 5	26,300 smp/sec
System 2 only (strategic)	5 / 5	24,890 smp/sec
Dual-process (S1 + S2)	5 / 5	26,300 smp/sec

reflexive pruning produces regime-ordered behavior (0/5 in both cases). Conversely, System 2 alone incurs a 5.3% throughput reduction (24,890 vs. 26,300 smp/sec) because without System 1's statistical pruning, the 32B model handles decisions that System 1 would resolve reflexively. The dual-process configuration achieves both goals simultaneously, confirming that the two systems are complementary rather than redundant.

4.2 Regime Coherence and Monotonicity

Table 2: Financial monotonicity checks under STABLE and STRESS Heston parameter anchors. "Raw" = model queried without controller. "With Controller" = dual-process controller with monotone projection applied. Both conditions use identical trained weights on 500 collocation points; the difference is solely the controller's presence.

Monotonicity Check	Raw Output	With Controller
Quote implied vol: STABLE → STRESS	×	✓
Quote price: STABLE → STRESS	×	✓
Surface mean vol: STABLE → STRESS	×	✓
Outlook shock-score: STABLE → STRESS	×	✓
Shock-risk labels ordered low to high	×	✓

Table 2 is the central finding: PDE residual minimization is a necessary but insufficient training objective for regime-ordered behavior. The raw model correctly minimizes $\mathcal{L}_{\text{PDE}}$; it fails all five monotonicity checks because the PDE loss imposes no constraint on the ordering of outputs across different Heston parameter configurations. This result is not a deficiency of KANs specifically; it is a property of any physics-constrained network trained without explicit regime-ordering terms in the objective. Heston parameters used: STABLE (κ=3.0, θ=0.02, σ=0.15, ρ=−0.50), STRESS (κ=0.5, θ=0.09, σ=0.60, ρ=−0.85), r=0.05, K=100, T=1.

4.3 Pricing Performance

Table 3: Replay price MAE by market regime. Lower is better. `spikan_plain_iv_hybrid` achieves best overall MAE (0.9317), with particular strength in transition and stress regimes where regime-conditioned corrections matter most.

Model	Stable	Transition	Stress	Overall
`bs_plain`	0.8398	2.1037	1.6355	1.4597
`spikan_plain_hybrid`	1.1542	2.4827	0.4745	1.3705
`spikan_plain_iv_hybrid`	1.1628	1.4622	0.4239	**0.9317**
`rf_direct`	8.3183	10.2679	1.8707	6.8190

Table 3 shows the IV-residual hybrid achieves the best overall replay MAE. The advantage is largest in transition and stress regimes—exactly the conditions where regime-conditioned

corrections matter. The random forest baseline performs poorly across all regimes, consistent with its inability to enforce PDE structure. Cross-asset results (preliminary, not shown) indicate the hybrid advantage is specific to macro-narrative assets (silver, oil) and absent on structural assets (semiconductors, technology), suggesting the architecture's domain fit signal rather than architectural failure.

4.4 Engineering Performance

Table 4: Engineering performance on NVIDIA H200, batch size 4,096. Zero queue overflows and zero GPU blocking validate the non-blocking coordination design.

Metric	Value
Sustained training throughput (5 epochs, 1M rows)	26,300 smp/sec
E2E microbenchmark throughput (50 steps, cold start)	6,506 smp/sec
System 1 (fast path) latency	<50 ms
System 2 (slow path) latency	200–500 ms
Decision queue overflows (1M-row stress test)	0
GPU blocking time from LLM	0 ms
Dual-process vs. baseline throughput delta	+4.4%

Table 4 shows the asynchronous non-blocking design achieves zero GPU blocking across a 1M-row stress test. In a hypothetical synchronous design, a single 500 ms System 2 decision would block ≈3,200 training steps at sustained throughput; the asynchronous design reduces this cost to zero. The observed 4.4% throughput gain over a pure gradient-descent baseline is directional and unablated; System 1 progressive pruning of near-zero edges is a candidate explanation (reduced computational graph load) but requires formal ablation.

5 Discussion and Conclusion

5.1 What the Results Show

Three statements can be made with confidence from the experimental results:

(1) The ablation confirms the division of labor. System 2 alone achieves regime-ordered behavior; System 1 alone preserves throughput; only the dual-process configuration achieves both. This is not just an efficiency tradeoff: it reflects the qualitative difference between statistical threshold decisions (System 1) and multi-step mathematical reasoning over PDE geometry (System 2).

(2) Monotonicity requires the controller—PDE training alone is insufficient. Five financial monotonicity checks all fail under the raw model with identical weights and pass only with the semantic controller. This isolates training-objective insufficiency as a property of the objective, not model capacity. Any physics-constrained network trained under PDE residuals alone may exhibit this failure under distributional shift.

(3) The IV-residual hybrid is best on narrative assets. The `spikan_plain_iv_hybrid` achieves the best overall replay MAE (0.9317) with particular strength in stress and transition regimes, confirming that the hybrid design captures regime-dependent pricing corrections that simpler baselines miss.

5.2 Limitations and Future Work

Current limitations include:

1. **Small protocol sample sizes:** protocol benchmark has $n \leq 15$ prompts per tier; rates are existence proofs, not stable probability estimates.

2. **Single quote date:** the live benchmark uses one quote date only; generalization across dates is unestablished.
3. **Prompt-engineered, not learned policy:** the controllers execute fixed system prompts with no gradient signal over LLM weights from downstream topology outcomes. LoRA fine-tuning infrastructure exists but awaits production trajectory data.
4. **Monotonicity is extrinsic:** regime ordering is enforced by the semantic controller, not embedded in the training objective. The path to making it intrinsic is ranking losses such as $\max(0, V_{\text{STABLE}} - V_{\text{STRESS}} + \epsilon)$.
5. **No cross-asset fine-tuning:** the architecture's advantage is specific to macro-narrative assets; separate PI-KANs per asset use canonical Heston parameter anchors, which may not fit assets with structurally different volatility dynamics.

Future work should close the gap from prompt-engineered to policy-learned controller via LoRA fine-tuning on logged agent trajectories, embed regime-ordering ranking losses directly in $\mathcal{L}$, and replicate the ablation at larger scale with formal confidence intervals.

In summary, KANs' edge-level symbolic replaceability creates a topology surgery problem that gradient descent cannot solve. The dual-process architecture addresses this by matching cognitive task type to model capability: fast statistical pattern recognition for high-frequency reflexive maintenance (System 1), slow mathematical reasoning for low-frequency strategic surgery (System 2). The central finding—that PDE training alone fails regime-ordering checks—may extend beyond options pricing to any physics-constrained network under distributional shift. The semantic controller is a structural remedy; ranking losses are the path to making monotonicity intrinsic.

References

Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21, 2019.

James D Hamilton. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2):357–384, 1989.

Blanka Horvath, Aitor Muguruza, and Mehdi Tomas. Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models. volume 21, pages 11–27, 2021.

Daniel Kahneman. *Thinking, Fast and Slow.* Farrar, Straus and Giroux, 2011.

Andrei Nikolaevich Kolmogorov. On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition. In *Doklady Akademii Nauk SSSR*, volume 114, pages 953–956. 1957.

Ziming Liu, Yixuan Wang, Vaibhav Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y Hou, and Max Tegmark. KAN: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756*, 2024.

Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessè, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.